New Horizons
Centro de Capacitación

Práctica: Aprobación Multinivel, Cierre en Tesorería y Trazabilidad Inmutable

Paso a paso — Implementar límites por monto, registrar pagos y generar el timeline forense

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 6 — APIs del flujo de aprobación
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Intermedio
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Implementar la matriz de límites de aprobación en el dominio (Jefatura, Gerencia, Directorio).
- Construir el endpoint POST /api/solicitudes/{id}/aprobar validando el límite del aprobador autenticado.
- Construir el endpoint POST /api/solicitudes/{id}/rechazar con motivo obligatorio.
- Construir el endpoint POST /api/solicitudes/{id}/pagar para liquidar la solicitud en Tesorería.
- Exponer el endpoint GET /api/solicitudes/{id}/timeline con el historial inmutable para la vista de detalle.

2. Desarrollo paso a paso

Paso 1 — Matriz de límites de autorización en el Dominio

En la entidad Usuario o servicio de dominio, validamos si el rol tiene atribución monetaria suficiente.

Usuario.cs

```
public decimal ObtenerLimiteAprobacion()
{
    if (Roles.Any(r => r.Nombre == "Directorio")) return decimal.MaxValue;
    if (Roles.Any(r => r.Nombre == "Gerente")) return 5000.00m;
    if (Roles.Any(r => r.Nombre == "Aprobador" || r.Nombre == "Jefatura")) return 1000.00m;
    return 0m;
}
```

Paso 2 — Endpoint de Aprobación jerárquica

En SolicitudService.cs implementamos AprobarAsync verificando límites y conflicto de intereses.

SolicitudService.cs

```
public async Task<ResultadoTransicionDto> AprobarAsync(int solicitudId, int aprobadorId, string?
comentario)
{
    var solicitud = await _uow.Solicitudes.ObtenerConDetallesAsync(solicitudId);
    var aprobador = await _uow.Usuarios.ObtenerConRolesAsync(aprobadorId);

    if (solicitud is null || aprobador is null)
        throw new KeyNotFoundException("Solicitud o usuario aprobador no encontrado.");

    // Validación de negocio en el Dominio
    solicitud.ValidarAutorizacionAprobar(aprobador);
    solicitud.Aprobar(aprobador.Id, comentario ?? "Aprobación conforme a políticas.");

    await _uow.CompletarAsync();

    return new ResultadoTransicionDto(solicitud.Id, solicitud.Estado.ToString(), "Solicitud
aprobada con éxito.");
}
```

Paso 3 — Endpoint de Rechazo con motivo

El rechazo es un estado final que requiere sustento explícito por escrito.

SolicitudService.cs

```
public async Task<ResultadoTransicionDto> RechazarAsync(int solicitudId, int responsableId,
string motivo)
{
    if (string.IsNullOrWhiteSpace(motivo))
        throw new ArgumentException("El motivo de rechazo es obligatorio.");

    var solicitud = await _uow.Solicitudes.ObtenerConDetallesAsync(solicitudId);
    if (solicitud is null) throw new KeyNotFoundException("Solicitud no encontrada.");

    solicitud.Rechazar(responsableId, motivo);
    await _uow.CompletarAsync();

    return new ResultadoTransicionDto(solicitud.Id, solicitud.Estado.ToString(), "Solicitud
rechazada.");
}
```

Paso 4 — Cierre en Tesorería y Movimiento de Pago

Tesorería registra el desembolso con el comprobante bancario, pasando el estado a Pagada.

SolicitudService.cs

```
public async Task<ResultadoTransicionDto> PagarAsync(int solicitudId, PagarSolicitudDto dto)
{
    var solicitud = await _uow.Solicitudes.ObtenerConDetallesAsync(solicitudId);
    if (solicitud is null) throw new KeyNotFoundException("Solicitud no encontrada.");

    if (solicitud.Estado != EstadoSolicitud.Aprobada)
        throw new ReglaDominioException("Solo se pueden pagar solicitudes previamente
aprobadas.");

    solicitud.RegistrarPago(dto.TesoreroId, dto.MedioPago, dto.ReferenciaOperacion,
dto.MontoPagado);
    await _uow.CompletarAsync();

    return new ResultadoTransicionDto(solicitud.Id, solicitud.Estado.ToString(), "Solicitud
pagada y liquidada.");
}
```

Paso 5 — Endpoint de Línea de Tiempo (Timeline)

Exponemos la consulta completa con el historial inmutable para la pantalla de trazabilidad.

SolicitudesController.cs

```
[HttpGet("{id:int}/timeline")]
public async Task<ActionResult<List<TimelinePasoDto>>> ObtenerTimeline(int id)
{
    var solicitud = await _uow.Solicitudes.ObtenerConDetallesAsync(id);
    if (solicitud is null) return NotFound();

    var pasos = solicitud.Aprobaciones
        .OrderBy(a => a.Fecha)
        .Select(a => new TimelinePasoDto(
            a.EstadoResultante.ToString(),
            a.Responsable != null ? a.Responsable.NombreCompleto : $"Usuario #{a.ResponsableId}",
            a.Fecha,
            a.Comentario))
        .ToList();

    return Ok(pasos);
}
```

3. Retos adicionales

1. Prueba el caso en que un usuario con rol Jefatura intenta aprobar una solicitud de S/ 2,500 y verifica que la API devuelva error de límite de monto.
2. Implementa concurrencia optimista con RowVersion en la entidad Solicitud para evitar dobles aprobaciones simultáneas.
3. Crea una prueba unitaria con xUnit que verifique el flujo completo: Borrador -> Enviada -> EnRevision -> Aprobada -> Pagada.
4. Verifica que una solicitud en estado Rechazada o Pagada no permita ninguna transición posterior.

4. Entregables

- Endpoints /aprobar, /rechazar, /pagar y /timeline operativos y probados vía REST.
- La base de datos almacena el registro de MovimientoTesoreria con medio de pago y número de operación.
- Historial completo de auditoría demostrable a través del endpoint de timeline.
- Pruebas de rechazo y aprobación multinivel validadas con xUnit.